

Track Document — Complete Engineering Flow

Project 05 — Dual-Core RV32I SoC with Coherent Memory Subsystem

Document ID: TRACK-RISCV-SOC-05

Owner: arafi

Purpose: This document maps the **complete technical track** from digital logic design fundamentals through RTL, synthesis, GDS, and FPGA implementation. It defines **what concepts to study/apply at each stage**, **what to pursue**, and **what the learning milestones are**. It is your roadmap through the entire flow.

1. Track Overview



Each stage has:

- **Concepts to deal with** — the theoretical/engineering knowledge you need.
- **What to pursue** — the concrete tasks and outputs.
- **Milestone/exit criteria** — what "done" means before moving to the next stage.

2. Stage 1 — Logic Design (LogSim / DLD)

Concepts to Deal With

Concept	Why it matters for this project
Combinational logic (gates, MUX, decoder, ALU)	The RV32I ALU is pure combinational logic. You must be able to design an ALU from gate-level thinking before writing RTL.
Sequential logic (flip-flops, registers, counters)	PC, register file, cache line storage, state machines — all sequential.
Finite State Machines (Moore/ Mealy)	The D-cache manager, coherence controller, AXI-Lite master/slave are all FSMs. This is the most critical concept.
Memory elements (SRAM, register arrays)	Cache tag/data, I-SRAM, shared SRAM are all memory. Understanding SRAM vs register-array trade-offs is key for both ASIC and FPGA.
Bus / interconnect concepts	Address decoding, arbitration, multi-master bus — needed for AXI-Lite interconnect.
Clocking & reset	Single-clock domain, async reset, synchronizer — the foundation of reliable RTL.
Timing (setup/hold, critical path)	Understanding the critical path through the ALU and cache determines if you hit 20 MHz.
Hazards & glitches	Avoiding combinational loops and glitchy signals in the coherence path.

What to Pursue

1. **Design the ALU logic on paper.** Write the truth table / gate logic for all RV32I ALU operations (ADD, SUB, AND, OR, XOR, SLT, SLL, SRL, SRA). Identify the critical path (SUB with SLT is typically longest).

2. **Design the cache FSM on paper.** Draw the state diagram for the D-cache manager (states: IDLE, CHECK, HIT, MISS, AXI_AR, AXI_R, FILL, AXI_AW, AXI_W, AXI_B, INVALIDATE). This is the heart of the system.
3. **Design the coherence controller FSM on paper.** Draw the state diagram for tracking I/S/M across two cores and dispatching invalidations.
4. **Design the arbiter logic.** A round-robin arbiter for 2 masters — draw the truth table and priority logic.
5. **Draw the system block diagram.** Show all modules, data paths, control signals. This becomes the RTL architecture.
6. **Sketch the memory map.** Decide addresses for shared SRAM, MMIO, UART, GPIO.

Exit Criteria

- ☐ ALU logic table complete (all 10+ operations)
- ☐ D-cache FSM state diagram drawn
- ☐ Coherence controller FSM state diagram drawn
- ☐ Arbiter logic designed
- ☐ System block diagram drawn
- ☐ Memory map defined

Transparency note: Skipping this stage and jumping to RTL is the #1 reason students produce buggy, un-synthesizable code. The FSM designs here are what you translate directly into SystemVerilog `always` blocks. Get them right on paper first.

3. Stage 2 — RTL Design (SystemVerilog)

Concepts to Deal With

Concept	Why it matters
SystemVerilog module structure	Clean, synthesizable module boundaries with defined port lists.
<code>always_ff</code> for sequential, <code>always_comb</code> for combinational	Avoid inferred latches — use the right block type.
Structured types (<code>struct packed</code> , <code>enum</code>)	Cache lines, coherence states, AXI channels benefit from typed design.
Parameterization	Cache size, line count, address widths should be parameterizable.
AXI4-Lite protocol	5-channel handshake (AW, W, B, AR, R). Valid/ready handshake on each channel. This is the bus backbone.
Memory modeling	Register arrays (<code>logic [31:0] mem [0:1023]</code>) vs macro instantiation.
Reset strategy	Async active-low, proper inference (<code>always_ff @(posedge clk or negedge rst_n)</code>).
Module hierarchy	Top-level instantiates and wires everything; no glue logic at top.

What to Pursue (build order)

3.1: alu.sv



3.2: reg_file.sv



3.3: control_unit.sv



3.4: rv32i_core.sv



3.5: sram_reg_arr
i_sram_shared_cri

Detailed build tasks:

1. **alu.sv** — Combinational ALU. Parameterized width (default 32). All RV32I operations. Zero/compare flags.
2. **reg_file.sv** — 32×32-bit register file. 2 read ports, 1 write port. x0 always zero. Synchronous write, combinational or sync read (decide based on single-cycle timing).
3. **control_unit.sv** — Instruction decoder. Decodes 32-bit instruction to control signals (ALU op, reg write, mem read/write, branch, etc.).
4. **rv32i_core.sv** — Integrate ALU + reg file + control unit + PC logic + data memory interface. Single-cycle: every instruction completes in 1 clock.
5. **sram_reg_array.sv** — Generic parameterized register array (depth, width configurable). Used for I-SRAM and shared SRAM. Synchronous read, synchronous write.
6. **d_cache.sv** — Cache line storage (valid, tag, data, state). Direct-mapped. Parameterized line count.
7. **d_cache_mgr.sv** — Cache controller FSM. Handles hit/miss, AXI-Lite master interface, coherence invalidation interface. **This is the most complex module — give it the most attention.**
8. **coherence_ctrl.sv** — Tracks per-line, per-core state. On write_notify from one core, invalidates the other core's line. Maintains status/counters for MMIO.
9. **axi_lite_arbiter.sv** — Round-robin arbiter for 2 masters. Grants bus access; ensures one transaction at a time per arbitration cycle.
10. **axi_lite_decoder.sv** — Address decode to route to shared SRAM / MMIO / UART / GPIO. Returns DECERR for unmapped.
11. **mmio_regs.sv** — AXI-Lite slave. Readable coherence status, event counters, doorbell, control.
12. **uart_core.sv** — AXI-Lite slave. TX with baud generator, RX with baud sampler. 8N1.
13. **gpio_led.sv** — Simple AXI-Lite slave, single output register for LEDs.
14. **riscv_soc_top.sv** — Integrate everything. Instantiate 2 cores, 2 I-SRAMs, 2 D-cache managers, 1 coherence controller, arbiter, decoder, shared SRAM, MMIO, UART, GPIO. Wire all signals. Reset synchronizer.
15. **Yosys smoke test** — Run Yosys synthesis on the top module ASAP (even before verification is complete). Catch unsynthesizable constructs early.

RTL Coding Standards

- Use **always_ff @(posedge clk or negedge rst_n)** for all flops.
- Use **always_comb** / **assign** for all combinational logic. Never **always @(*)** without **comb**.
- No **initial** blocks in synthesizable RTL (only in testbenches and memory init wrappers).
- No **\$display**, **\$finish** in synthesizable code.
- No delays (**#10**) in synthesizable code.
- Every signal has a defined reset value.
- Parameters for all widths/depths — no magic numbers.

Exit Criteria

- ☐ All RTL modules written and individually compileable
- ☐ **rv32i_core** passes a simple instruction test in isolation
- ☐ **d_cache_mgr** passes a simple hit/miss test
- ☐ **coherence_ctrl** passes a simple invalidate test

- ☐ `riscv_soc_top` compiles and Yosys synthesizes (smoke test)
- ☐ No inferred latches (check Yosys warnings)

4. Stage 3 — Functional Verification

Concepts to Deal With

Concept	Why it matters
Testbench architecture	Directed TB (self-checking) + UVM (constrained-random). Two layers of verification.
AXI4-Lite BFM (bus functional models)	Drive AXI-Lite transactions in the testbench without manually toggling signals.
Self-checking testbench patterns	Expected-value comparison, <code>\$error</code> / <code>\$fatal</code> on mismatch, pass/fail flag.
UVM methodology	Agents, sequences, scoreboard, monitor, coverage. Object-oriented SV.
Functional coverage	Coverpoints for coherence states, events, hit/miss, error.
Code coverage	Line, toggle, branch, FSM state coverage. Identify untested code.
Assertions (SVA)	Immediate/concurrent assertions for protocol checking (AXI handshake, coherence invariants).
Constrained-random verification	Randomize addresses, data, timing to stress the design.

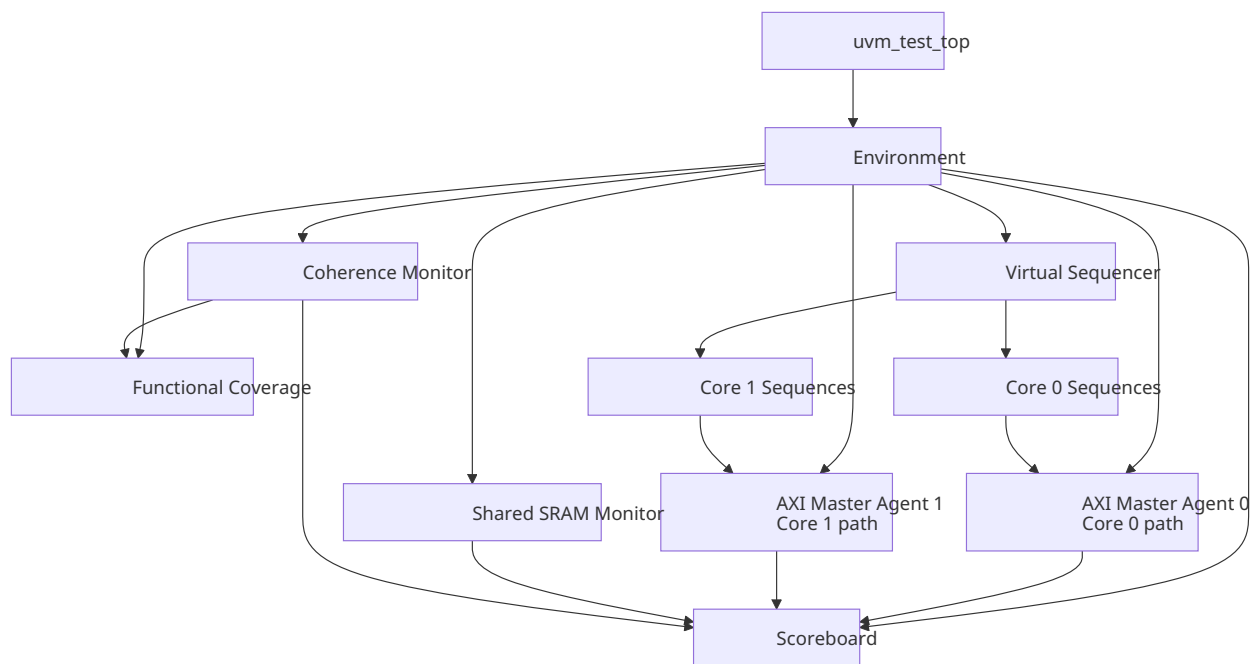
What to Pursue

4.1 Directed Self-Checking Testbench (`tb_directed.sv`):

Test cases in order:

1. **Reset test:** Assert `rst_n=0`, check all state = I, PC = RESET_PC, counters = 0.
2. **Single-core memory access:** Core 0 writes shared addr, reads back, asserts value matches.
3. **Cache hit:** Core 0 loads addr A twice; second access is a hit (check hit counter increments).
4. **Cache miss:** Core 0 loads addr A first time; miss counter increments.
5. **Cross-core coherence (THE key test):** Core 0 writes addr A; check Core 1's line for A becomes I; Core 1 reads A, gets new value.
6. **Stale data prevention:** Verify Core 1 does NOT get old value after invalidation.
7. **Simultaneous write:** Both cores write same addr same cycle; check deterministic winner; final SRAM value correct.
8. **No-cached-copy:** Core 0 writes addr that Core 1 never loaded; no invalidation needed; check no spurious invalidate.
9. **Error/unmapped access:** Core accesses unmapped address; check DECERR/SLVERR response.
10. **Counter verification:** Check INV_COUNT, HIT_COUNT, MISS_COUNT match expected after test sequence.

4.2 UVM Environment:



UVM components to build:

- **axi_lite_master_agent** — driver + sequencer + monitor. Drives AXI-Lite transactions.
- **axi_lite_seq_item** — transaction object (addr, data, we, wmask, response).
- **core0_seq**, **core1_seq** — constrained-random read/write sequences to shared addresses.
- **coherence_vseq** — virtual sequence coordinating both cores for coherence scenarios.
- **shared_sram_scoreboard** — predicts expected SRAM content; compares with monitor observations.
- **coherence_monitor** — observes invalidation events; checks: every cross-core write must produce an invalidation.
- **coverage_collector** — covergroups for: coherence state per line per core, hit/miss, invalidation count, simultaneous-write, error response.
- **axi_lite_if** — SV interface connecting UVM to DUT.

Assertions (SVA) to include:

```

// AXI-Lite handshake: valid stays until ready
assert property (@(posedge clk) disable iff (!rst_n)
  awvalid |-> awvalid until awready);
// Coherence invariant: after write_notify, inv_req must fire within 2 cycles
assert property (@(posedge clk) disable iff (!rst_n)
  write_notify |-> ##[1:2] inv_req);
// No two masters granted simultaneously
assert property (@(posedge clk) disable iff (!rst_n)
  !(grant0 && grant1));

```

Exit Criteria

- ☐ Directed TB: all 10 test cases pass, self-checking (no manual waveform inspection needed)
- ☐ UVM: scoreboard passes for 1000+ random transactions

- ☐ Functional coverage at least 90% on coherence states, hit/miss, invalidation, error
- ☐ Code coverage at least 90% line, at least 80% toggle/branch
- ☐ All assertions pass (0 failures)

5. Stage 4 — Synthesis (Yosys via OpenLane)

Concepts to Deal With

Concept	Why it matters
Technology mapping	Yosys maps your RTL to sky130 standard cells. Understanding this helps debug synthesis failures.
SDC constraints	Clock definition, I/O delays. Without a clock constraint, timing analysis is meaningless.
Synthesis warnings/errors	Inferred latches, undriven signals, multi-driven nets — fix ALL before proceeding.
Netlist vs RTL	The synthesized netlist is a gate-level representation. It may behave differently if your RTL had sim-only constructs.
Area optimization	Yosys can optimize; understanding <code>abc</code> (the tech mapper) helps.
Hierarchy / flatten	Yosys may flatten or preserve hierarchy. Know which modules are black-boxed (SRAM macros if used).

What to Pursue

- Prepare OpenLane config:** `config.json` with design name, top module, clock period, PDK (sky130A), standard cell library (`sky130_fd_sc_hd`), verilog files list.
- Run Yosys synthesis** via OpenLane (or standalone Yosys first for a smoke test).
- Check synthesis report:**
 - Number of cells, area estimate.
 - Warnings/errors — fix ALL (especially inferred latches).
 - Unmapped cells — any cell Yosys couldn't map to sky130? Investigate.
- Gate-level netlist simulation (optional but recommended):** Simulate the synthesized netlist with a few test cases to confirm synthesis didn't break functionality.
- Write SDC** (see TRD). Ensure clock is defined; add I/O delays.
- Iterate:** Fix RTL issues, re-synth, re-check until clean.

OpenLane config.json key fields:

```
{
  "DESIGN_NAME": "riscv_soc_top",
  "VERILOG_FILES": ["dir/rtl/**/*.sv"],
  "CLOCK_PERIOD": 50,
  "CLOCK_PORT": "clk",
  "CLOCK_NET": "clk",
  "CORE_UTILIZATION": 0.55,
  "FP_SIZING": "relative",
  "PL_TARGET_DENSITY": 0.6,
  "PDK": "sky130A",
  "STD_CELL_LIBRARY": "sky130_fd_sc_hd"
```



```
}
```

Exit Criteria

- ☐ Yosys synthesis completes with 0 errors
- ☐ 0 inferred latches (check warnings)
- ☐ All cells mapped to sky130_fd_sc_hd
- ☐ Gate-level netlist simulates (smoke test) correctly
- ☐ SDC file written and accepted by synthesis

6. Stage 5 — Physical Design (OpenROAD via OpenLane to GDS)

Concepts to Deal With

Concept	Why it matters
Floorplan	Core area, utilization, pin placement, macro placement (if using OpenRAM). Too high utilization causes routing congestion. Too low wastes area.
Power planning (PDN)	Power ring + stripes on metal layers. Without adequate PDN, IR drop causes timing failures.
Placement	Standard cell placement. Congestion-aware. OpenROAD places cells to minimize wirelength.
Clock tree synthesis (CTS)	Balanced clock distribution to all flops. Minimizes clock skew. Critical for timing.
Routing	Signal routing on metal layers. DRC-clean routing is the goal.
Timing analysis (STA)	Setup/hold checks against the clock. WNS (Worst Negative Slack), TNS (Total Negative Slack).
Congestion analysis	Routing overflow on metal layers. High congestion means unroutable design.
Metal stack	sky130 metal layers (M1-M5/M6). Which layer for std cell routing, which for power, which for signal.

What to Pursue

The OpenLane flow runs these stages automatically. Your job is to configure, monitor, and fix issues:

1. Floorplan stage:

- Set core utilization to ~55% (leave room for routing).
- If using SRAM macros: place macros first, then let OpenROAD place std cells.
- Check: core area reasonable, pin placement OK.

2. Power plan stage:

- OpenLane generates a power grid automatically (VDD/VSS rings + stripes).
- Check: grid coverage adequate, no IR-drop hotspots (OpenROAD reports this).

3. Placement stage:

- Check: placement congestion, placement density per region.
- If congestion > 95% in any region, increase core area or reduce utilization.

4. CTS stage:

- Check: clock skew (should be less than ~10% of clock period), max transition.
- If skew too high, adjust CTS buffers/config.

5. Routing stage:

- Check: DRC violations (must be 0), routing congestion.
- If DRC violations, fix (usually means floorplan or placement issue).

6. Sign-off stage:

- Run DRC (Magic) — 0 violations required.
- Run LVS (Netgen) — netlist match required.
- Capture layout screenshot (KLayout GDS view).

Physical Design Reports to Capture

Report	What it shows	Where in OpenLane
Area report	Total cell area, number of cells	Synthesis / final
Timing report	WNS/TNS (setup and hold)	After routing (STA)
Power report	Internal/switching/leakage power	After routing
Congestion report	Routing overflow per layer	After routing
DRC report	Design rule violations	Sign-off (Magic)
LVS report	Netlist vs layout match	Sign-off (Netgen)
Layout screenshot	Visual GDS view	KLayout

Exit Criteria

- ☐ OpenLane flow completes end-to-end
- ☐ DRC: 0 violations
- ☐ LVS: clean (netlist match)
- ☐ Timing: WNS ≥ 0 (setup met), hold met
- ☐ All reports captured
- ☐ Layout screenshot captured (KLayout)

Transparency note: If timing doesn't close at 20 MHz, check the critical path. In a single-cycle RV32I core, the critical path is almost certainly ALU to register file write or the cache tag comparison to AXI path. If it's the ALU, consider lowering the clock target. If it's the cache/coherence path, check for unnecessary combinational depth.

7. Stage 6 — Sign-off (DRC / LVS)

Concepts to Deal With

Concept	Why it matters
DRC (Design Rule Check)	Verifies layout geometry against PDK rules (min width, spacing, enclosure). Magic runs this in OpenLane.

Concept	Why it matters
LVS (Layout Versus Schematic)	Verifies the GDS layout netlist matches the synthesized schematic netlist. Netgen runs this.
GDS extraction	Extracting the netlist from the layout for LVS comparison.
Antenna rules	Long metal traces can accumulate charge during fabrication, causing gate damage. OpenLane checks this.

What to Pursue

1. After OpenLane completes, check the sign-off logs.
2. **DRC:** If violations exist, they're usually fixable by adjusting floorplan/routing. Common: metal spacing violations, antenna violations.
3. **LVS:** If mismatch, usually means a net was optimized or renamed during synthesis. Compare netlists to find the discrepancy.
4. **KLayout:** Open the final GDS in KLayout for a visual screenshot — this is a required deliverable.
5. If using OpenRAM macros, ensure the macro GDS is included in the merge step.

Exit Criteria

- ☐ DRC: 0 violations
- ☐ LVS: clean
- ☐ KLayout layout screenshot captured
- ☐ Sign-off reports saved

8. Stage 7 — FPGA Implementation

Concepts to Deal With

Concept	Why it matters
FPGA synthesis (Xilinx Vivado)	Different from ASIC synthesis. Vivado maps to LUTs/FFs/BRAMs/DSPs.
BRAM inference	I-SRAM and shared SRAM should infer BRAM (not distributed LUT RAM) for efficient area.
Clock management (MMCM/PLL)	Board reference clock (100/125 MHz) to MMCM to 50 MHz core clock.
Constraints file (.xdc)	Pin assignments, clock constraints, I/O standards (LVCMOS33).
Bitstream generation	Vivado generates the <code>.bit</code> file for programming.
Board programming	Program via JTAG (Arty: onboard programmer; Zybo: onboard programmer).
UART on FPGA	Connect UART TX to board USB-UART bridge pins.

What to Pursue

1. Create Vivado project:

- Add RTL source files.
- Set top module.
- Add constraints (.xdc).
- Select target device (Arty: xc7a100t-1csg324; Zybo: xc7z010/xc7z020).

2. Write constraints (.xdc):

```
# Clock input (Arty 100 MHz)
set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports clk_in]
create_clock -name sys_clk -period 10.0 [get_ports clk_in]
# Reset (active-low, use a button)
set_property -dict { PACKAGE_PIN C12 IOSTANDARD LVCMOS33 } [get_ports rst_n]
# UART
set_property -dict { PACKAGE_PIN D10 IOSTANDARD LVCMOS33 } [get_ports uart_tx]
set_property -dict { PACKAGE_PIN A9 IOSTANDARD LVCMOS33 } [get_ports uart_rx]
# LEDs
set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports {led[0]}]
set_property -dict { PACKAGE_PIN K15 IOSTANDARD LVCMOS33 } [get_ports {led[1]}]
# ... (check board docs for exact pins)
```

3. Add MMCM/PLL wrapper:

- Instantiate Xilinx MMCM primitive to generate 50 MHz from board reference.
- Or use the Clocking Wizard IP.

4. I-SRAM / shared SRAM init:

- For FPGA, use `$readmemh` in an initial block (Vivado synthesizes this to BRAM init).
- Or use `readmemh` in a wrapper that synthesizes to BRAM init values.

5. Synthesize + implement + generate bitstream.

6. Program the board and run the demo:

- Open a serial terminal (115200 baud).
- Observe UART output: core activity, coherence events, shared-data before/after values.
- Observe LEDs: heartbeat + event indicators.

FPGA Demo Protocol (what to show)

```
UART Output Example:
=== RISC-V Coherent SoC Demo ===
Core 0: Writing 0xDEAD to addr 0x0000
Core 0: Write complete
Coherence: INVALIDATE core 1 line 0
Core 1: Reading addr 0x0000
Core 1: Value = 0xDEAD [PASS - updated data visible]
Core 1: Old cached = 0xBEEF [showing stale was invalidated]
=== Demo Complete ===
```

Exit Criteria

- ☐ Vivado synthesis + implementation + bitstream generation succeeds
- ☐ Timing constraints met on FPGA (no negative slack)
- ☐ Board programmed and running
- ☐ UART output shows correct coherence demo sequence

- ☐ LEDs show core activity + coherence events
- ☐ Updated data visible to core 1 after invalidation (verified via UART)

9. Stage 8 — Documentation & Delivery

Concepts to Deal With

Concept	Why it matters
Architecture document	Required deliverable. Must explain the design to a reader who didn't build it.
Report compilation	Synthesis, physical-design, verification, FPGA reports — all must be organized and referenced.
Reproducibility	Another engineer should be able to rebuild from your repo.

What to Pursue

Architecture document (`architecture.md`):

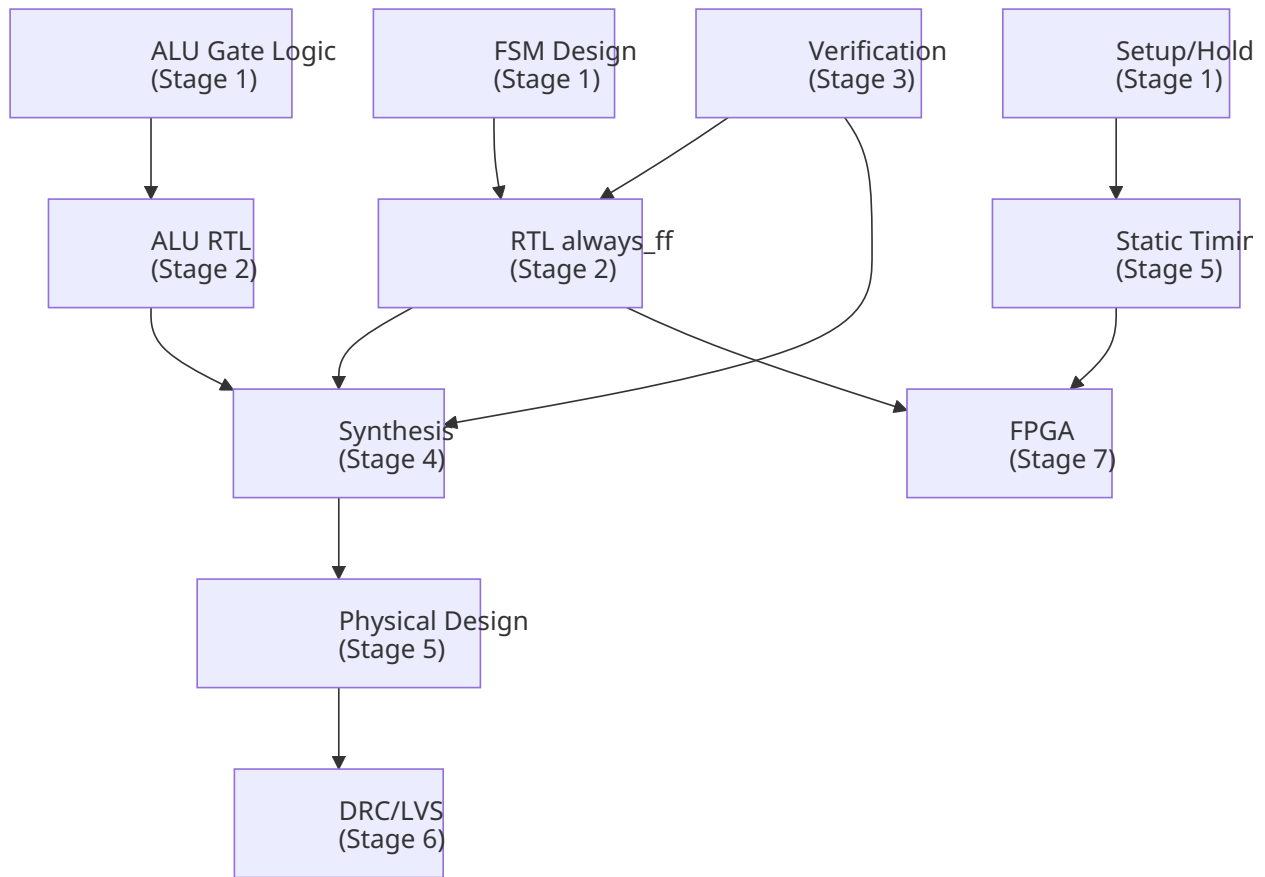
1. System overview (block diagram)
2. Module descriptions (each module's function, interface, key design decisions)
3. Coherence protocol explanation (I/S/M states, invalidation mechanism)
4. Memory map
5. AXI4-Lite interface description
6. Verification approach (directed + UVM)
7. Physical design results (area, timing, power, DRC/LVS)
8. FPGA demo description
9. Known limitations / future improvements

Exit Criteria

- ☐ Architecture document complete
- ☐ All deliverables from PRD checked off
- ☐ Repo is clean, organized, and documented
- ☐ Reports organized and readable

10. Cross-Stage Concept Map

This map shows which concepts connect across stages — understanding these links prevents "siloed" thinking.



Key insight: A mistake in Stage 1 (e.g., wrong FSM design) propagates to Stage 2 (wrong RTL), Stage 3 (test failures), Stage 4 (synthesis issues), and Stage 5 (timing). **Fixing concepts in Stage 1 is 10x cheaper than fixing them in Stage 5.** This is why the brief says "understand it completely" first.

11. Common Pitfalls by Stage

Stage	Pitfall	Consequence	Prevention
1	FSM states not exhaustive	Unreachable states, hang	Draw complete state diagram, handle ALL transitions
1	Not defining reset values	Unknown state after reset	Specify reset value for every register
2	Inferred latches	Synthesis failure, timing issues	Use <code>always_comb</code> , ensure all branches assign all outputs
2	Combinational loop	Simulator hangs, synthesis fails	No logic feedback without a register
2	Multi-driven signals	X in sim, synthesis error	Each signal driven by exactly one <code>always</code> block
3	TB not self-checking	Rely on waveform eyeballing	Use <code>assert</code> / <code>\$error</code> , pass/fail flag
3	UVM over-engineering	Scope creep, time loss	Minimal agents, keep sequences simple
4	Leaving synth to last day	No time to fix RTL issues	Yosys smoke test on Day 3
5	Utilization too high	Routing congestion, DRC fails	Keep ~55% utilization
5	No clock constraint	Meaningless timing	Always define clock in SDC

Stage	Pitfall	Consequence	Prevention
6	Macro GDS not merged	LVS mismatch	Include all macro GDS in OpenLane config
7	Wrong pin assignments	No output on board	Verify pins from board schematics
7	No BRAM init	Program doesn't run	Use <code>\$readmemh</code> synthesisable init

12. Learning Outcomes (what you should know after completing this track)

After completing this project, you will have demonstrated understanding of:

- ☐ RISC-V ISA and single-cycle processor design
- ☐ Cache hierarchy fundamentals (direct-mapped cache, hit/miss, write-through)
- ☐ Cache coherence concepts (I/S/M states, invalidation-based coherence)
- ☐ Bus protocols (AXI4-Lite: 5-channel handshake, arbitration, address decoding)
- ☐ SystemVerilog RTL design (synthesizable coding, FSMs, structured types)
- ☐ Verification methodology (directed TB, UVM, coverage, assertions)
- ☐ ASIC synthesis flow (Yosys, technology mapping, SDC)
- ☐ ASIC physical design flow (floorplan, PDN, placement, CTS, routing, sign-off)
- ☐ DRC/LVS sign-off
- ☐ FPGA implementation flow (Vivado, BRAM, MMCM, constraints, bitstream)
- ☐ System integration (multi-core, shared memory, peripherals)
- ☐ Documentation and reproducibility

This is a **full-stack IC design project** — from logic gates to silicon to working FPGA hardware. The coherence subsystem is what makes it more than a "two cores and a memory" exercise.